HikeAlong

Project Documentation

Subject: Software Design
Student: Butuza Raisa Briana

1. Requirements Analysis

1.1 Assignment Specification

The objective of this project is to develop a full-stack application for managing an **event reservation system**.

The application allows:

- Users to browse events and reserve seats
- Organizers to create and manage events

Each user has a defined role (**USER** or **ORGANIZER**) which determines the actions they are allowed to perform.

The system is implemented using:

- **Spring Boot** for the backend
- **Angular** for the frontend

The application ensures:

- Data integrity
- Input validation
- Business rule enforcement
- Clear error handling

1.2 Functional Requirements

The application has the following functional requirements:

- **User Management**
  - Users can register, log in, and manage their profile
- **Role Management**
  - Support role-based access control (USER / ORGANIZER)
- **Event Management**
  - Organizers can:
    - Create events
    - View events
    - Update events
    - Delete events
- **Reservation Management**
  - Users can:
    - Browse available events

        - Reserve seats
- **Input Validation**
    - Validate:
        - Name format
        - Email format
        - Password strength
        - Event details (title, date, location, capacity)
- **Duplicate Prevention**
    - Prevent multiple accounts with the same email
- **Authorization Rules**
    - Only organizers can manage events
- **Error Handling**
    - Display clear and user-friendly error messages

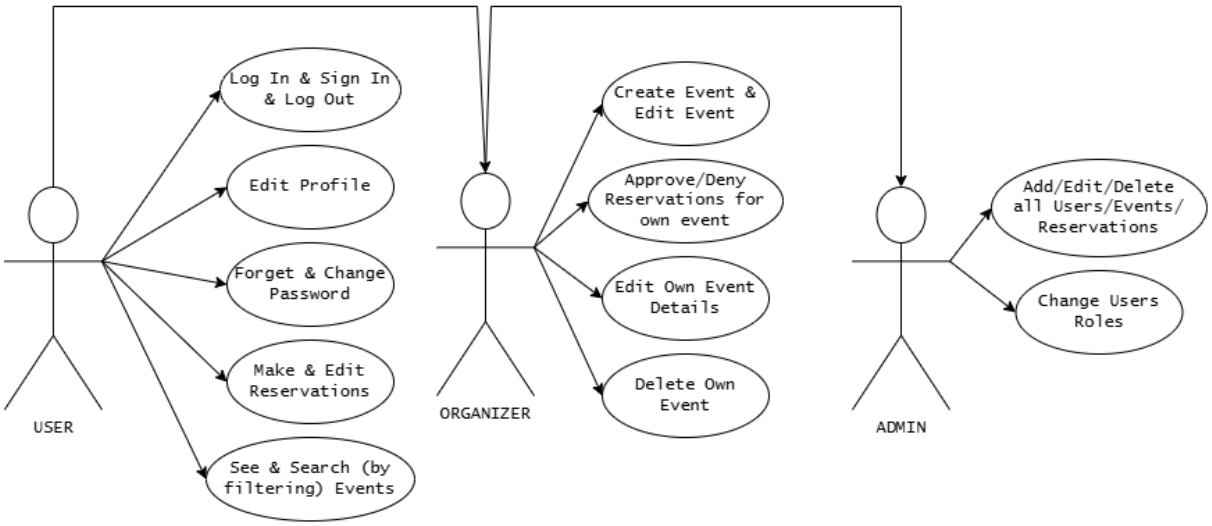
1.3 Non-functional Requirements

The application has the following non-functional requirements:

The application has the following non-functional requirements:

- **Layered Architecture**
    - Controller layer
    - Service layer
    - Repository layer
- **ORM Integration**
    - Use **Spring Data JPA**
- **Dependency Injection**
    - Use Spring DI container
- **Global Exception Handling**
    - Centralized error handling with meaningful HTTP responses
- **Scalability & Maintainability**
    - Modular and extensible design
- **Data Integrity**
    - Ensure consistency of reservations and event capacity
- **Security**
    - Authentication and role-based authorization

2. Use-Case Model



Basic User

- Creates account
- Edit profile details
- Recuperate password with email
- Browser or search by filtering events
- Makes, sees and edits own reservations

Organizer

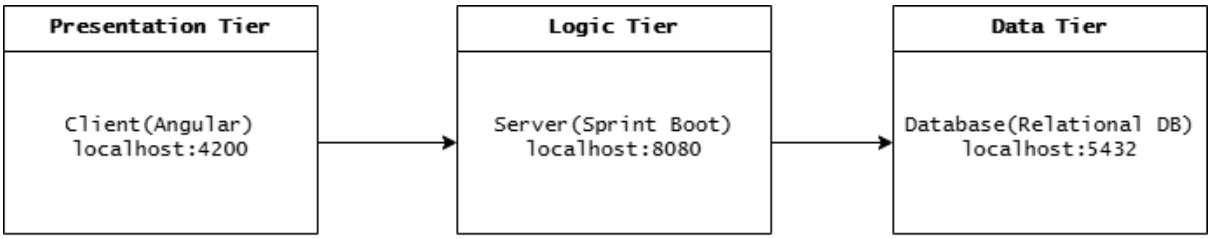
- Inherits all basic user
- Views, creates, edits, deletes own events
- Views, edits, accepts or denies reservations from own events

Admin

- Can do everything a basic user and all organizers can do
- Change all users roles

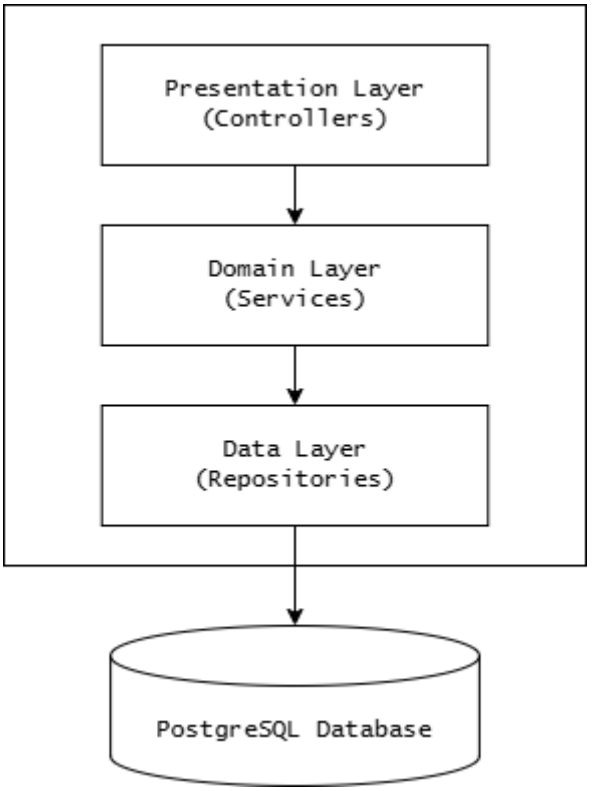
3. System Architecture Design

3.1 Architectural Pattern Description



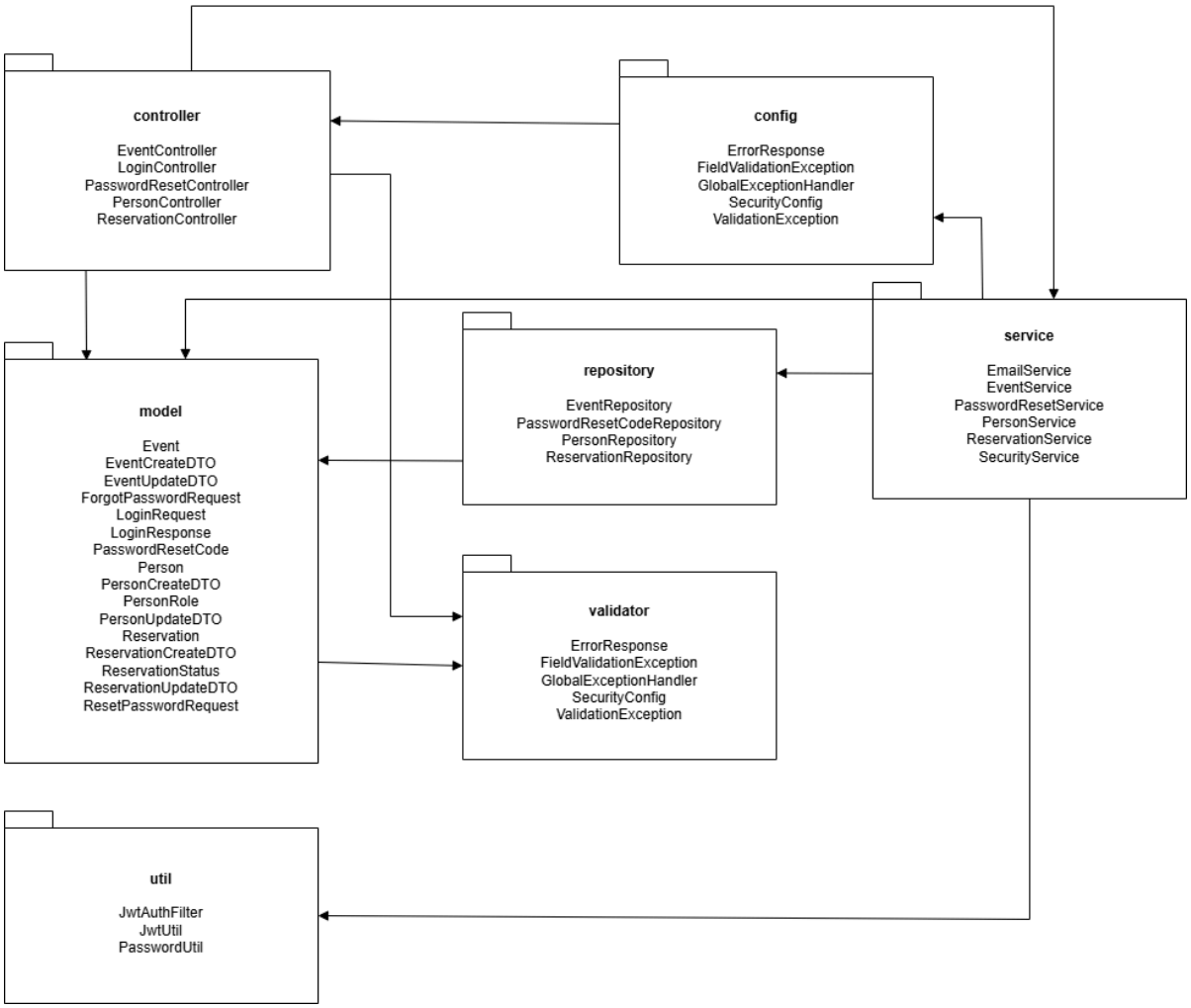
- **Presentation Tier (Client):** Implemented as a dynamic frontend using Angular. It handles user interactions, sends requests to the backend, and displays clear, user-friendly error messages when a request fails due to validation or server errors.
- **Logic Tier (Server):** Implemented as a robust backend with Spring Boot. This tier manages the core application logic, relying on a Dependency Injection (DI) container to manage component life cycles. It enforces strict input validation and features a centralized Global Exception Handler to capture business logic failures and map them to appropriate HTTP responses.
- **Data Tier (Database):** Uses a relational database integrated with Spring Data JPA (ORM) to handle all database interactions and entity mapping

3.2 Layered Architecture



- **Presentation Layer (Controllers):** The entry point for requests (e.g., PersonController).
- **Domain Layer (Services):** Where the business logic resides (e.g., ReservationtService, EventService).
- **Data Layer (Repositories):** Handles the actual communication with the database using Spring Data JPA.

3.3 Package + Class Diagram



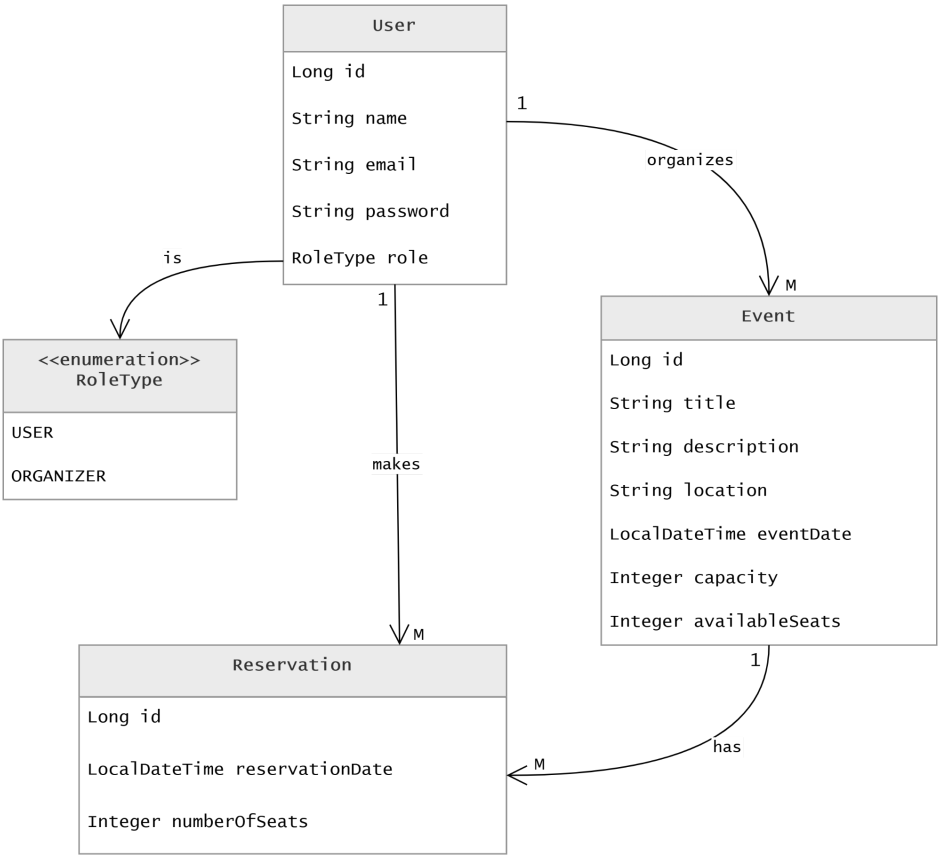
The application's backend architecture is organized into several key modules that enforce a clean separation of concerns. The modules are structured as follows:

- **Controller Layer:**

  This package contains the REST Controllers (PersonController, CourseController, DepartmentController, AssignmentController, SubmissionController, LoginController, and PasswordResetController) which act as the entry point for requests from the presentation tier. These controllers map HTTP verbs to specific backend logic and delegate tasks to the appropriate services.

- **Service Layer:**
  This module contains the business logic implementations (PersonService, EventService, ReservationService, SecurityService, PasswordResetService and EmailService). It acts as an intermediary, processing data, handling authentication checks, facilitating multi-step flows (like email-based password recovery), and managing transactions before data is sent to the repository.
- **Repository Layer:**
  This module manages data access through Spring Data JPA interfaces such as PersonRepository, EventRepository, ReservationRepository and PasswordResetTokenRepository. These interact directly with the PostgreSQL database. (Note: Authentication and credential verification rely directly on the PersonRepository rather than a separate credentials repository)
- **Model Layer:**
  This central package defines the domain entities (Person, Course, Department, Assignment, Submission, PasswordResetToken) and the Data Transfer Objects (DTOs) used for communication. Specifically for authentication, it includes:
  - LoginRequest: A DTO containing the user's credentials (email and password) sent from the client.
  - LoginResponse: A DTO returned upon successful authentication, containing the user's assigned Role (admin, student, or professor), the generated JWT token, and the user's ID.
- **Config, Util & Validator:**
  - **Config:** Contains global handlers like GlobalExceptionHandler to manage application-wide error responses. It also houses the security backbone, including SecurityConfig, JwtAuthFilter, JwtService, and PasswordEncoderConfig to manage stateless JWT authentication and CORS policies.
  - **Util:** Contains helper classes like PasswordUtil to handle password hashing operations securely for the service layer.
  - **Validator:** Contains custom validation logic, such as StrongPasswordValidator, to ensure data integrity and security before records are persisted to the database.

3.4 Class Relation



Person ↔ Event (One-to-Many)

A Person with the role ORGANIZER can create many Events, but each Event has exactly one organizer. This is a one-to-many relationship mapped via the `organizer_id` foreign key in the Event table. In JPA, the Event entity holds a `@ManyToOne` reference to Person.

Person ↔ Reservation (One-to-Many)

A single Person (with role USER) can have many Reservations — one per event they want to attend. Each Reservation belongs to exactly one Person. This is again a one-to-many, mapped by the `person_id` foreign key in the Reservation table.

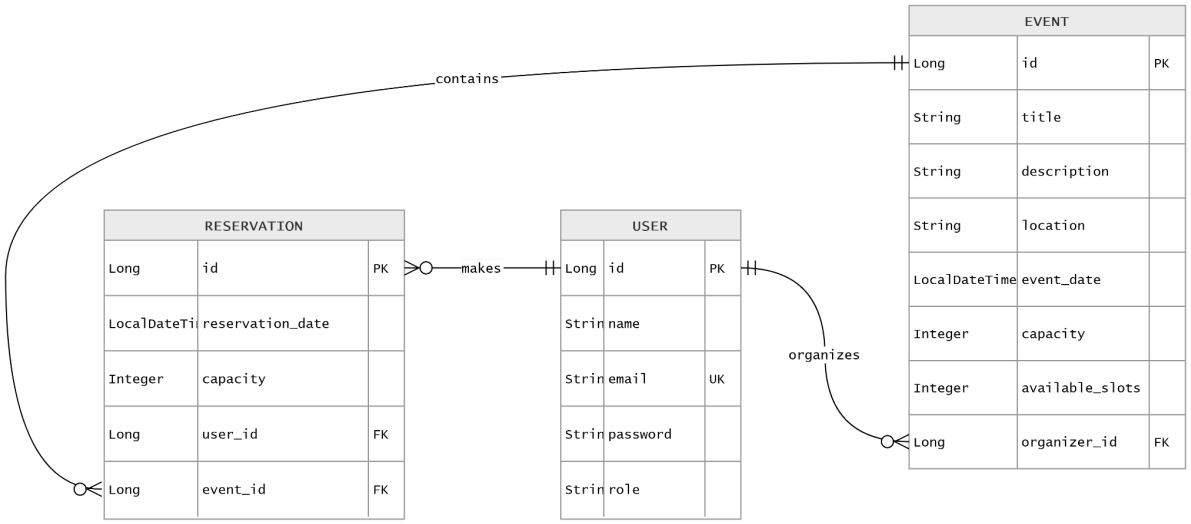
Event ↔ Reservation (One-to-Many)

A single Event can have many Reservations from different users. Each Reservation is tied to exactly one Event via the `event_id` foreign key. This is a one-to-many relationship.

Person ↔ Event (Many-to-Many through Reservation)

While there is no direct many-to-many mapping in the code, Person and Event are effectively connected in a many-to-many fashion through the Reservation join entity. A Person can reserve spots at many Events, and an Event can have many Persons reserving spots. The Reservation table acts as the associative/junction table, enriched with extra fields: `spots_reserved` and `status` (PENDING, ACCEPTED, DECLINED).

3.5 Data Model



User

- id - unique identifier
- name - full name of the user
- email - unique email address
- password - encrypted password
- role - role of the user (USER or ORGANIZER)

Event

- id - unique identifier
- title - name of the event
- description - event details
- location - place where the event will take place
- eventDate - scheduled date and time
- capacity - maximum number of seats
- availableSeats - remaining number of seats
- organizer - the user who created the event

Reservation

- id - unique identifier
- reservationDate - date and time when the reservation was made
- numberOfSlots - number of reserved seats
- user - the user who made the reservation
- event - the reserved event

4. Tests on Backend

Controller Integration Tests

PersonControllerIntegrationTests

- testGetPeople
- testAddPerson_ValidPayload
- testAddPerson_InvalidPayload
- testGetPersonById
- testGetPersonByEmail
- testPatchPerson_ValidPayload
- testDeletePerson
- testAddPerson_DuplicateEmail

EventControllerIntegrationTests

- testGetEvents
- testAddEvent_ValidPayload
- testAddEvent_InvalidPastDate
- testAddEvent_InvalidCapacity
- testGetEventById
- testPatchEvent_ValidPayload
- testDeleteEvent

ReservationControllerIntegrationTests

- testGetReservations
- testAddReservation_ValidPayload
- testAddReservation_InvalidSpotsReserved
- testGetReservationById
- testPatchReservation_ValidPayload
- testDeleteReservation

Service Unit Tests

PersonServiceTests

- testGetPeople
- testGetPersonById_ExistingId
- testGetPersonById_MissingId
- testGetPersonByEmail_ExistingEmail
- testGetPersonByEmail_MissingEmail
- testAddPerson_ValidPayload
- testAddPerson_DuplicateEmail
- testUpdatePerson_ExistingId
- testUpdatePerson_MissingId
- testPatchPerson_ValidPayload
- testPatchPerson_DuplicateEmail
- testDeletePerson
- 1testDeletePerson_MissingId

EventServiceTests

- testGetEvents
- testGetEventById_ExistingId
- testGetEventById_MissingId
- testAddEvent_ValidPayload
- testAddEvent_MissingOrganizer
- testUpdateEvent_ExistingId
- testUpdateEvent_MissingId
- testPatchEvent_ValidPayload
- testPatchEvent_WithOrganizerId
- testPatchEvent_MissingEventId
- testDeleteEvent
- testDeleteEvent_MissingId

ReservationServiceTests

- testGetReservations
- testGetReservationById_ExistingId
- testGetReservationById_MissingId
- testAddReservation_ValidPayload
- testAddReservation_MissingPerson
- testAddReservation_MissingEvent
- testUpdateReservation_ExistingId
- testUpdateReservation_MissingId

- testPatchReservation_ValidPayload
- testPatchReservation_MissingId
- testDeleteReservation
- testDeleteReservation_MissingId

SecurityServiceTests

- testLoginSuccess
- testLoginWrongPassword
- testLoginEmailNotFound